Billing Software

Developer: Akshat Shah

Status: v0.1 feature-complete (backend, frontend and automated tests)

Description

A billing software for an accountancy firm to manage clients, services and bills.

Features (v0.1)

- Login and logout (no public sign-up: an admin creates the staff accounts)
- Staff management (admin): create accounts, reset passwords, deactivate / reactivate
- Add client, update client details (including **Referred by**), delete client (soft delete, with restore)
- A **folder per client on the PC**, named after the client, holding their details and a printable copy of every bill (updated automatically; see "Client folders" below)
- Add service (with a description), update service, deactivate service
- Create bill with several line items and a live total preview. Each line shows the service's description as entered in the service, copied onto the bill
- Update bill
- Search bills by client name or bill number, newest first, paginated, with a **financial year dropdown** (Indian FY, April to March, e.g. 2026-27)
- Print-friendly bill view (prints without the browser's date, title and address header)
- Organisation settings (admin): firm details, **logo upload**, **payment details** (bank account, IFSC, UPI ID and instructions, so clients know where to pay), and a GST on/off toggle with a configurable rate. The logo and payment details print on every bill and in the client-folder copies

Tech Stack

Area	Technology
Language	TypeScript
Frontend	React (with Vite), React Router
Backend	Node.js + Express 5, Zod validation
Database	PostgreSQL (plain SQL migrations, `pg` driver)
Auth	JWT + bcrypt, roles: admin / staff
Tests	Vitest + Supertest
Version control	Git + GitHub
Project management	GitHub Projects

Project structure

```
billing-software/
├── server/              Express API
│   ├── migrations/      SQL files, applied in order at startup
│   ├── src/routes/      auth, users, clients, services, settings, bills
│   ├── src/lib/         money maths, paging, error handling
│   ├── src/scripts/     seed.ts (creates the first admin), restore.ts (restores a backup)
│   └── test/            automated tests
├── client/              React app
│   └── src/pages/       one file per screen
├── installer/           Windows installer (Inno Setup script, build script, wizard text)
├── .github/workflows/   CI tests and the release build
└── docs/decisions.md    why things are the way they are
```

Documentation

Document	For
User guide	Staff and administrators: every screen, step by step, backups and troubleshooting
API reference	Every endpoint, field limit, status code and the GST/financial-year rules
Architecture and development	How it is built, data model, configuration, tests, installer and releases
Decision records	Why each important choice was made

PDF copies of all of these are in `docs/pdf`: one PDF per document, plus `Billing-Software-Documentation.pdf`, which has everything in one file with a cover page and clickable contents. They are built from the Markdown files, so after editing the docs regenerate them (needs Microsoft Edge or Google Chrome installed):

```
cd docs/pdf-build
npm install
npm run build
```

Installing on a new computer (Windows)

For an office, use the installer. It needs nothing installed beforehand.

1. Open the project's **Releases** page on GitHub and download `BillingSoftware-Setup-<version>.exe` (the `.sha256` file next to it lets you verify the download).
2. Run it. Windows may say "unknown publisher" because the installer is not code-signed: choose **More info > Run anyway**.
3. The wizard explains what it will do, then asks for a network port (default 4000) and the first administrator's name, username and password. It shows a full summary before it installs anything.
4. When it finishes, the software is already running as a Windows service. Open `http://localhost:4000` on that PC, and `http://<that-pc-name>:4000` from the other office PCs.

What the installer sets up: the program in `C:\Program Files\Billing Software`, a Windows service that starts with Windows, a firewall rule for private/domain networks only, and a data folder `C:\ProgramData\BillingSoftware` holding the database, daily backups (last 14 kept) and logs. Running the

installer again upgrades the program and keeps all data. Restoring a backup: `"C:\Program Files\Billing Software\tools\restore.cmd" "<backup file>"` from an administrator prompt. The wizard's own pages (`before`, `after`) cover this in more detail.

Publishing a new installer (for the developer)

```
git tag v0.1.0
git push origin v0.1.0
```

The workflow in `.github/workflows/release.yml` runs the tests, builds the app, bundles Node.js and the service wrapper, compiles the installer with Inno Setup and attaches it to a GitHub Release. To build locally, run `installer\build.ps1`, then compile `installer\billing.iss` with Inno Setup 6.3+.

Getting started (for development)

You need Node.js 22 or newer. For a real database you also need PostgreSQL.

1. Backend

```
cd server
npm install
cp .env.example .env
```

Open `server/.env` and set:

- `DATABASE_URL`. Use your PostgreSQL connection string, for example `postgres://postgres:YOURPASSWORD@localhost:5432/billing` (create an empty `billing` database first). The default, `pglite:./.data`, is an embedded Postgres that needs no setup, which is handy for a quick try.
- `JWT_SECRET`. Any long random string.
- `ADMIN_USERNAME` / `ADMIN_PASSWORD`. The first admin account. Change the password from the example.

Then create the tables and the first admin, and start the API:

```
npm run seed
npm run dev
```

The API runs on http://localhost:4000.

2. Frontend

In a second terminal:

```
cd client
npm install
npm run dev
```

Open http://localhost:5173 and log in with the admin you seeded.

3. Tests

```
cd server
npm test
```

The tests use an in-memory Postgres (PGlite), so they need no database setup.

Running as one server

`npm run build` in `client/` creates `client/dist`. When that folder exists, the Express server serves it too, so the whole app runs from http://localhost:4000.

Key design rules

- **Money is whole paise** (integer), never floating point. The screens convert to rupees.
- **GST is a snapshot.** A bill stores the rate and amount it was created with. Changing the setting later never changes old bills, and editing a bill recalculates with that bill's own rate.
- **Soft delete** for clients and staff, so old bills always point at a real record.
- A bill line keeps a **copy** of the service's description, so editing the service later never changes old bills.
- A bill belongs to the **financial year** of its date (1 April to 31 March): 31 Mar 2026 is in FY 2025-26, 1 Apr 2026 is in FY 2026-27.
- Each bill records who created it and who last edited it, and when.
- Deactivated staff lose access immediately, not when their login token expires.

See *docs/decisions.md* for the reasoning behind these choices.

Look and feel

- **Light, dark and system themes.** A three-way switch in the top bar (and on the login page): *device setting*, *light*, *dark*. The choice is remembered in the browser and applied before the page paints, so there is no white flash. Bills stay a white sheet of paper in dark mode, exactly as they print, so any logo reads correctly.
- **Built on the principles in emilkowalski/skills**: restrained motion (this is a tool used all day, so only buttons, fades for messages and colour changes move, and nothing animates on a keyboard action), buttons that press down slightly, hover effects only on devices that can hover, semi-transparent edges instead of solid borders, tabular figures for amounts, tighter heading spacing, and a stress test with worst-case data.
- **Phone friendly.** The Bills list and the bill form become card layouts on narrow screens, inputs are 16px so iPhones do not zoom in, tap targets are at least 44px on touch screens, and safe-area insets are respected.
- **Plain-language errors and enforced limits.** Forms stop you typing past a field's limit, and messages read like "Phone is too long (at most 20 characters)".

Client folders

For every client the server keeps a normal folder, named after the client, as a human-readable second copy:

```
Client Files/
  Rajesh Patel/
    client-details.txt        details and a list of the client's bills
    client-data.json          the same data in full, for other programs
    bills/INV-000007_2026-11-05.html     each bill, printable in any browser
```

- Updated whenever a client or bill is saved, and checked again every time the server starts.
- Two clients with the same name: the second folder gets the client number added, e.g. `Rajesh Patel (2)`.
- Renaming a client renames the folder. Deleting a client keeps the folder and marks its details file as deleted. Folders are never deleted by the software.
- Location: `CLIENT_FILES_DIR` in `server/.env` (default `server/client-files` in development, the folder chosen in the installer wizard when installed). Admins can see the location, the last error, and press **Rebuild all client folders now** under Settings.
- If the disk cannot be written to, bills still save and Settings shows the error.

Known limitations (ideas for v0.2)

- Bill numbers come from a database sequence, so a bill that fails to save can leave a gap in the numbers (GST invoice numbers normally must be consecutive).
- Issued bills can be edited. A real accounting workflow would cancel them or issue credit notes instead.
- GST is one flat rate (no CGST/SGST/IGST split, HSN/SAC codes or GST reports).
- No PDF or email export (the browser's Print works).
- The login rate limit is in memory, so it resets when the server restarts.